

■ Design Payment System (Stripe)

System Design Interview | Financial Systems

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Process payments (card/bank/wallet); Refunds; Webhooks for status updates; Multi-currency; Fraud detection; Idempotency
Non-Functional	Exactly-once processing (no double charge); ACID guarantees; PCI DSS compliance; 99.999% availability; Audit trail
Scale	Stripe processes \$800B/year; Global routing; <300ms payment confirmation; Handle bank outages gracefully

■ High-Level Architecture

- Payment API → idempotency check → Payment Processor → Risk/Fraud engine → PSP (Visa/Mastercard network) → Bank
- State machine: PENDING → PROCESSING → COMPLETED/FAILED; each transition persisted to DB before action taken
- Two-phase commit with external systems: reserve funds → confirm; if confirm fails → release reservation
- Ledger DB: append-only, immutable double-entry bookkeeping; every financial event recorded permanently

■ Exactly-Once Payment Processing

- **Idempotency Key:** Client sends unique key per payment attempt; server stores key → result mapping; duplicate request returns cached result without reprocessing
- **Idempotency Store:** Redis (fast check) + PostgreSQL (persistent); key expires after 24h; critical for retry safety
- **DB Transaction:** Within a single service: use DB ACID transaction; INSERT payment + UPDATE balance atomically; rollback on failure
- **Two-Phase Commit:** Across services: phase 1 prepare (both services vote yes/no); phase 2 commit/abort; slow but necessary for cross-service money moves
- **Saga Pattern:** Alternative: sequence of local transactions with compensating transactions for rollback; better availability than 2PC

■ Ledger Design (Double-Entry Bookkeeping)

- Every transaction creates two ledger entries: debit one account, credit another; sum of all entries = 0
- Entries table: entry_id, transaction_id, account_id, amount, currency, direction(DR/CR), created_at — APPEND ONLY
- Balance = sum of all entries for account; recomputable from ledger; never stored as mutable field
- Immutability: no UPDATE or DELETE; corrections via reversal entries; complete audit trail

■ Fraud Detection

- Rule engine: real-time rules (velocity checks: >5 transactions/min from same card → flag; amount >\$10K → review)
- ML model: transaction features (amount, merchant category, location, time, device fingerprint) → risk score
- Risk score thresholds: <20 auto-approve; 20-70 review; >70 auto-decline; model updated daily
- 3D Secure (3DS): for high-risk transactions, redirect to bank for additional authentication (OTP, biometric)

■ Distributed Transaction Patterns

Two-Phase Commit (2PC)	Saga Pattern
✓ Strong consistency	✓ Higher availability
✓ All-or-nothing atomicity	✓ Non-blocking
✓ Blocking protocol	✓ Requires compensating transactions
✓ Lower availability (coordinator SPOF)	✓ Eventual consistency

■ PCI DSS Compliance

- Tokenization: card numbers replaced with tokens; actual PAN stored only in vault (isolated, encrypted environment)
- Network segmentation: cardholder data environment (CDE) isolated from rest of infrastructure
- Encryption in transit (TLS 1.2+) and at rest (AES-256); key rotation policy
- Audit logging: every access to cardholder data logged; logs immutable and retained per compliance requirements